



M



ARTIFICIAL INTELLIGENCE (Q) – APRIL 2026
KECERDASAN TIRUAN (Q) – APRIL 2026

Introduction on Machine learning and Deep learning

Mohammad Iqbal, S.Si., M.Si., Ph. D.



OUTLINES



- ❑ Machine Learning
- ❑ Deep Learning



Machine Learning



Artificial Intelligence...



Internet of Things

Artificial
Intelligence

Robotic

Data warehouse

Data Mining

Preprocessing data

SVM

Machine

LR

MLP

Learning

GPR

Deep CNN

GAN

Deep

Deep LSTM

GNN

Learning

VAE

Overview Machine Learning

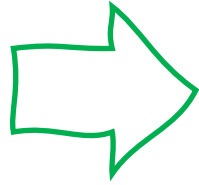


**Training
phase**



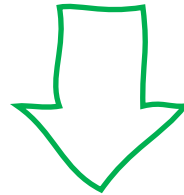
training data

ML algorithm

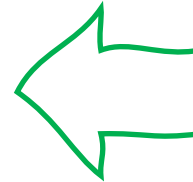


**Testing
phase**

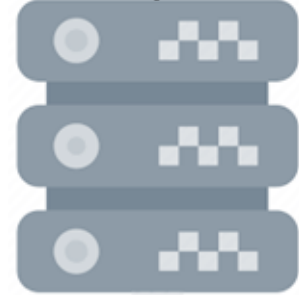
evaluation:
accuracy,
error



ML model



testing data



Major Task: Supervised Learning



Supervised learning

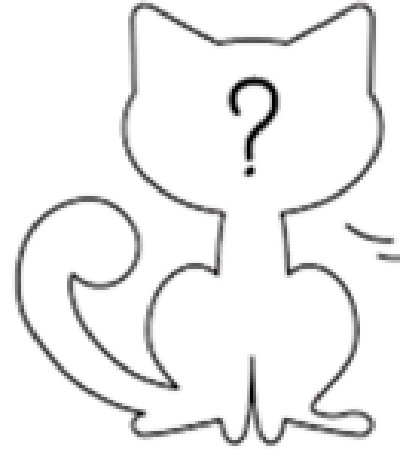
Data : $\{ (x_i, y_i) \mid x, y \in \mathbb{R} \}$

$x \in X$: data, $y \in Y$: label/class

Goal : learn from mapping

$$f: X \rightarrow Y$$

where Y is discrete or nominal values



This is a **CAT**

Major Task: Unsupervised Learning

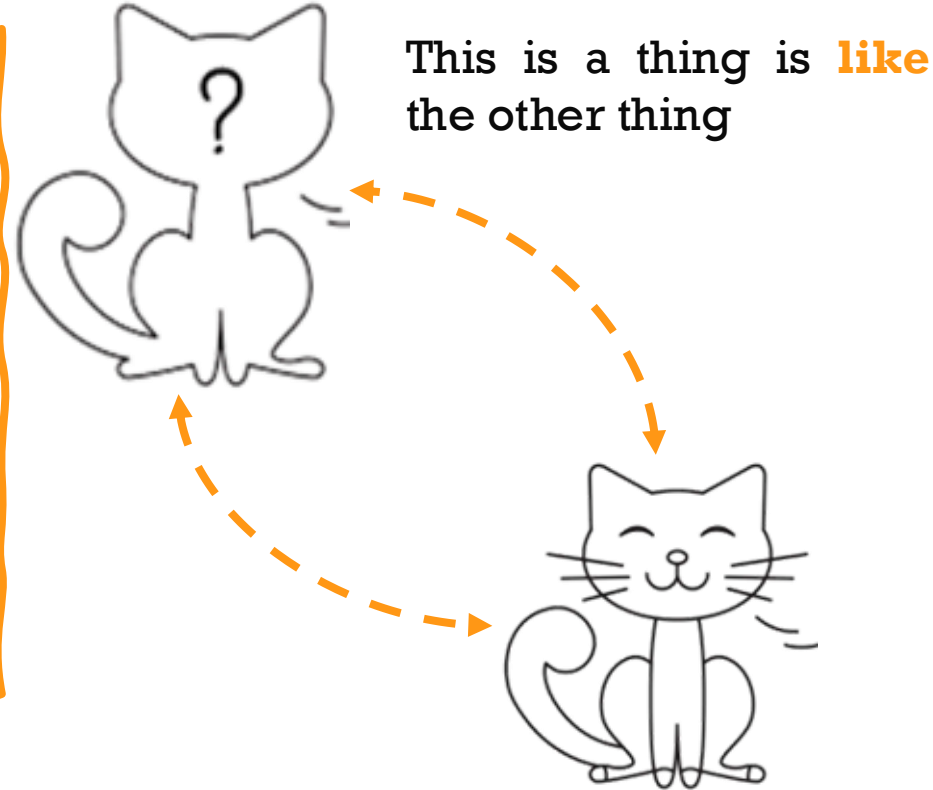


Unsupervised Learning

Data : $\{x_i \mid x \in \mathbb{R}\}$

$x \in X$: data (unlabeled)

Goal : learn hidden information / structure of data



Major Task: Reinforcement Learning



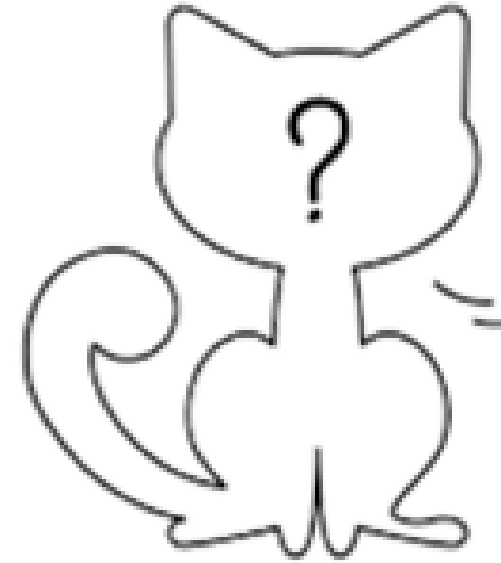
Reinforcement learning

Data : $\{(s, a), s \in S, a \in A\}$

$s \in S$: state, $a \in A$: action

Goal : maximize reward over the step

$$\max r(s_t, a_t)$$



This is thing **can make you happy**

Tasks



Novice

Master

Supervised learning



Augmented
Supervised learning



Semi Supervised learning



Reinforcement learning



Unsupervised learning



Dataset	Description
Training Data	: A set of example used for learning a model.
Validation Data	: A set of example can be used to tune model parameter (helps control overfitting) but cannot be used for learning the model.
Testing Data	: - A set of examples used to asses the performance of the final model and provide an estimation of the test error. - Note: Cannot be used to tune parameter or revise the model

Deep Learning

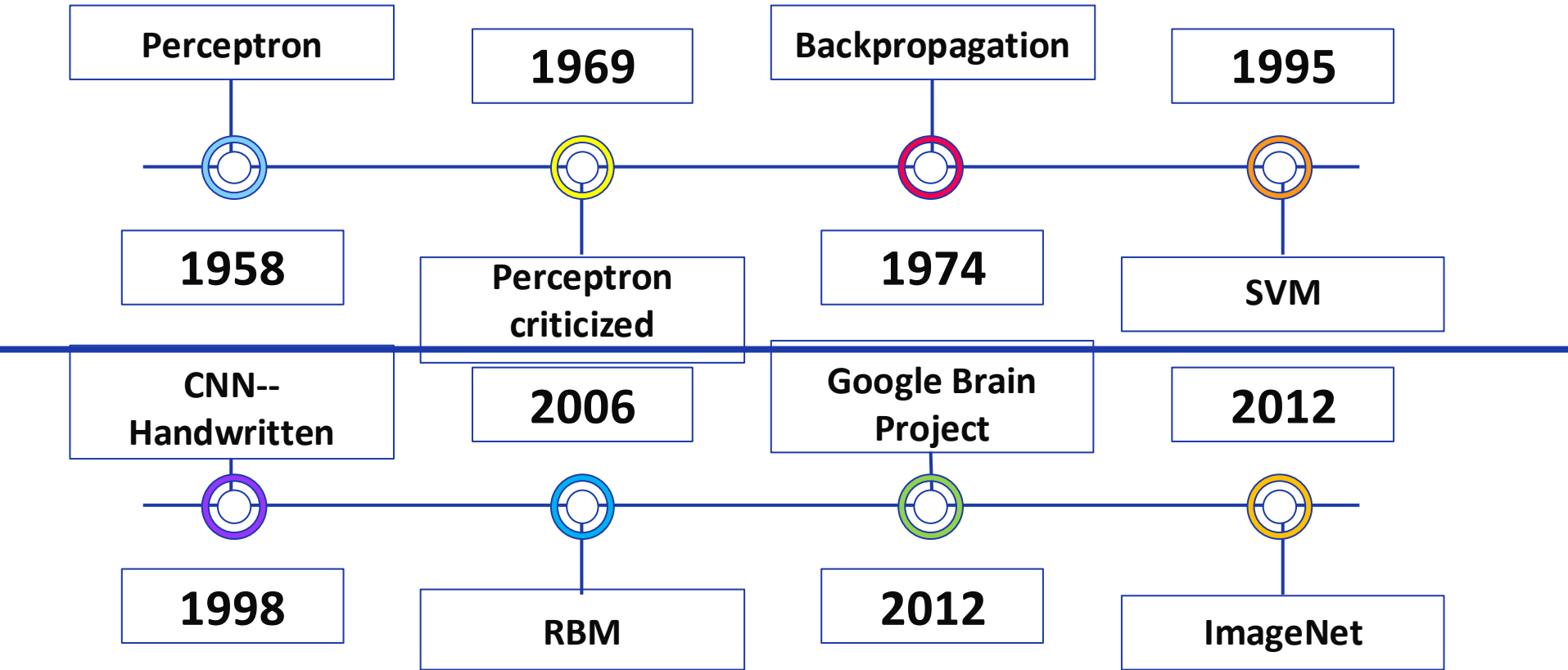


What is deep learning?

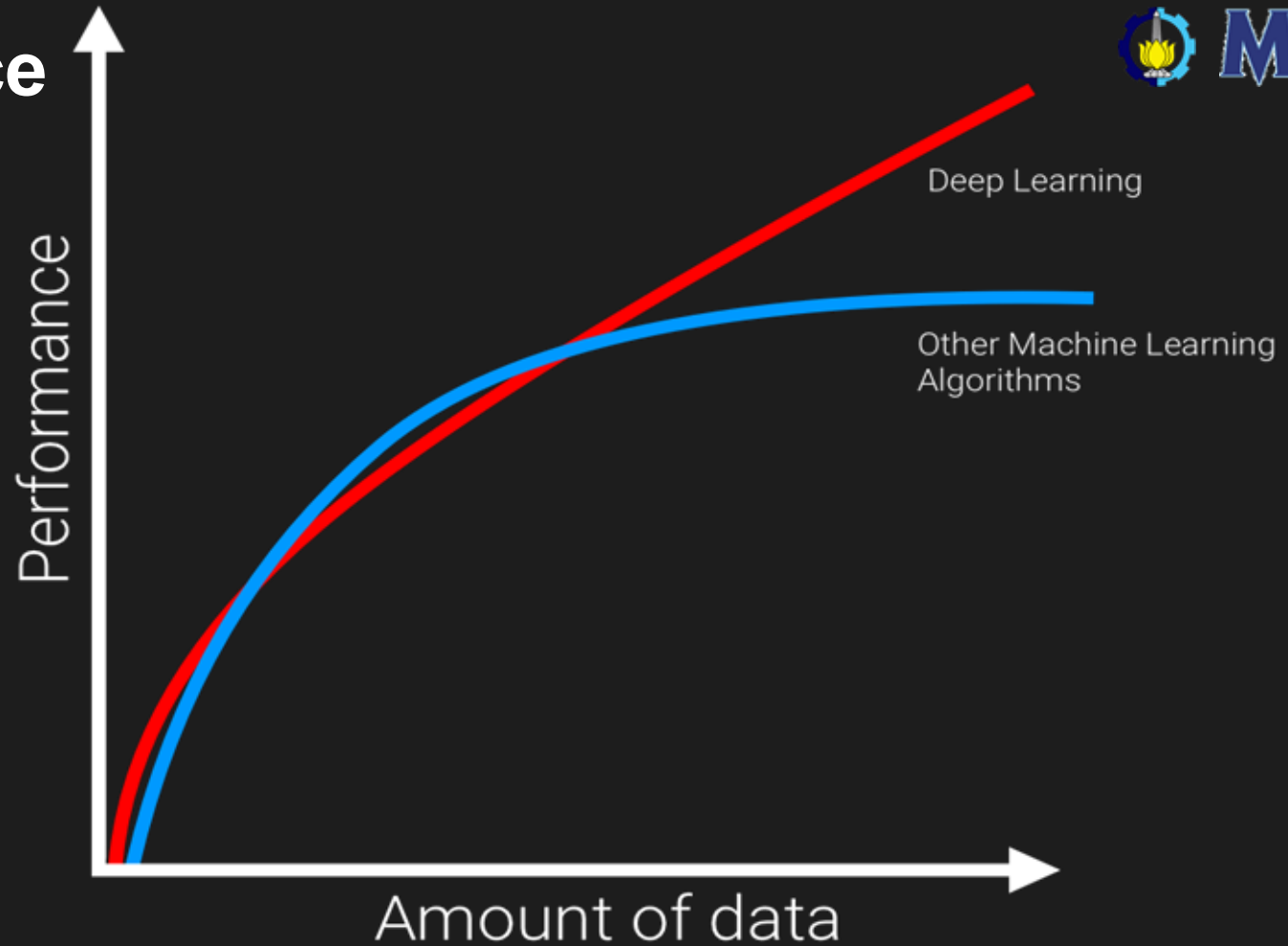


- ❑ Deep Learning is a part of machine learning which useful to **automatically create data representation**
- ❑ Deep learning also called **representation learning**

The history...



The difference



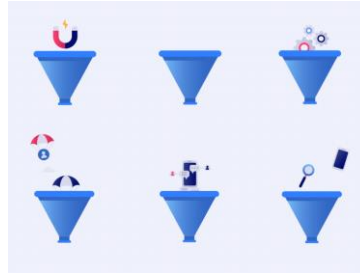
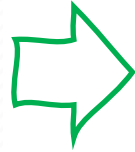
Machine Learning vs Deep Learning



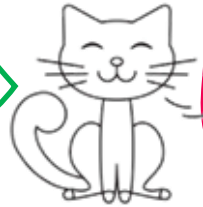
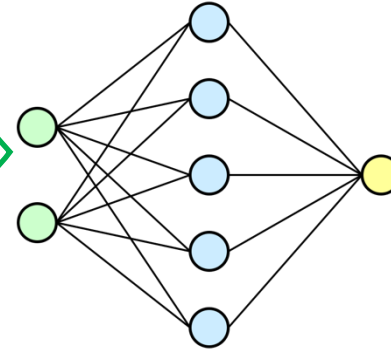
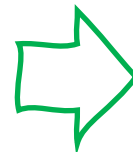
Machine Learning



input



feature extraction

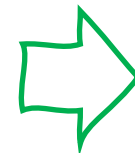
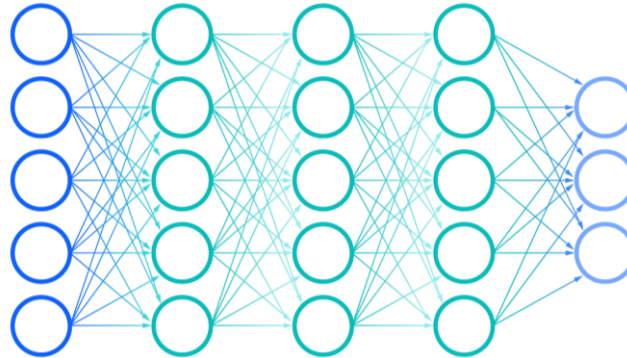
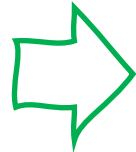


output

Deep Learning

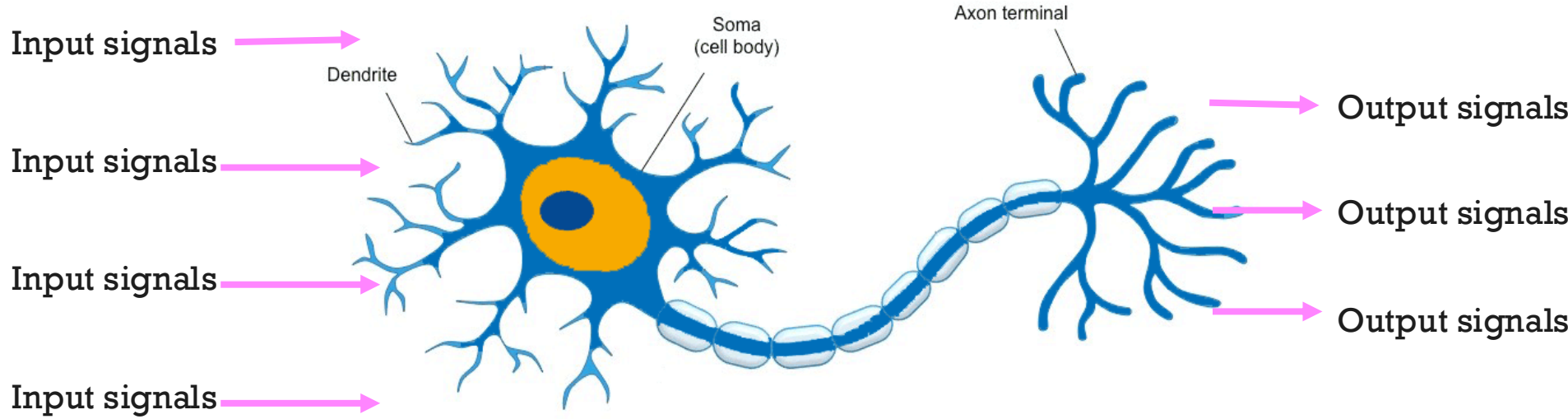


input



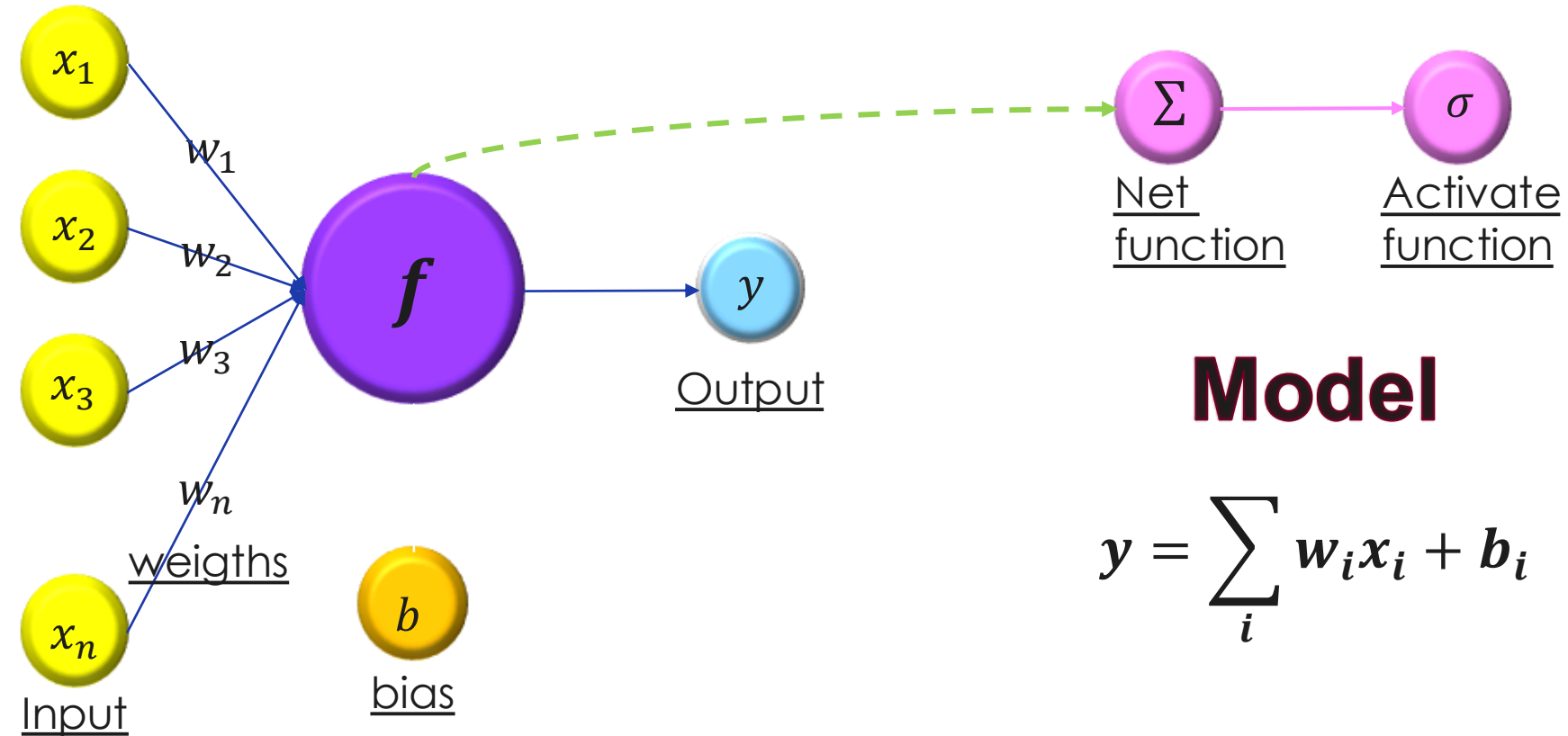
output

Perceptron concept



<https://adatis.co.uk/introduction-to-artificial-neural-networks/>

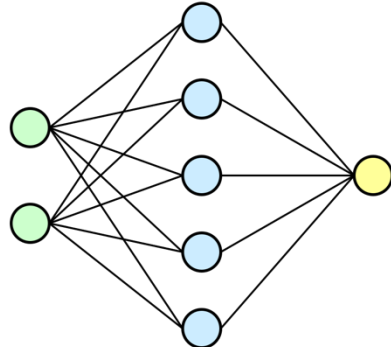
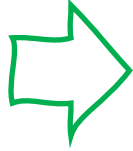
Single Layer Perceptron



Model

$$y = \sum_i w_i x_i + b_i$$

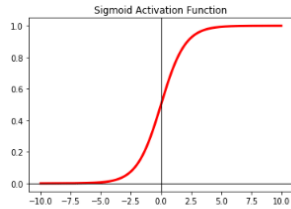
Activation function



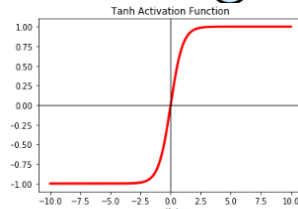
$$\text{softmax} \frac{e^{y_c}}{\sum_{c \in \mathcal{C}} e^{y_c}}$$



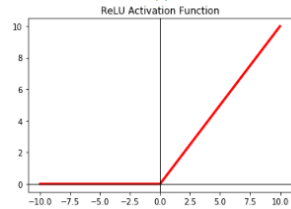
$$\begin{bmatrix} 0.7 \\ 0.1 \\ 0.2 \end{bmatrix} \begin{matrix} \text{cat} \\ \text{bird} \\ \text{dog} \end{matrix}$$



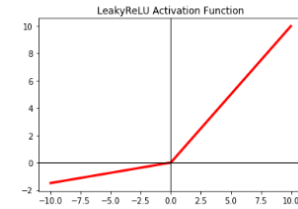
(a)



(b)



(c)

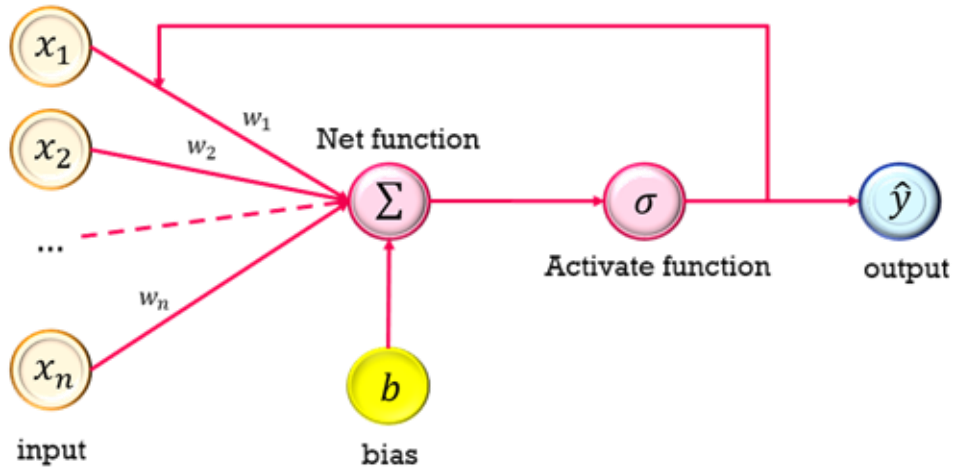


(d)

MLE :

$$p_{\theta}(C|x) \propto \text{argmax} \frac{e^{y_c}}{\sum_{c \in \mathcal{C}} e^{y_c}}$$

Perceptron Learning Rules



$$w_{new} = w_{new} + \Delta w$$
$$\Delta w = \alpha \cdot (y - h(x)) \cdot x$$

steps:

1. Initialize weights randomly;
2. If the output is correct $y = h(x)$, then the weights are remained;
3. If the output is incorrect $y \neq h(x)$, then the weights are changed that closer to y ;
4. Till converges when:
 - a. Linearly separable
 - b. α (learning rate) sufficiently small

But...



Perceptron learning rules

Failed to converges
Cannot linear separable

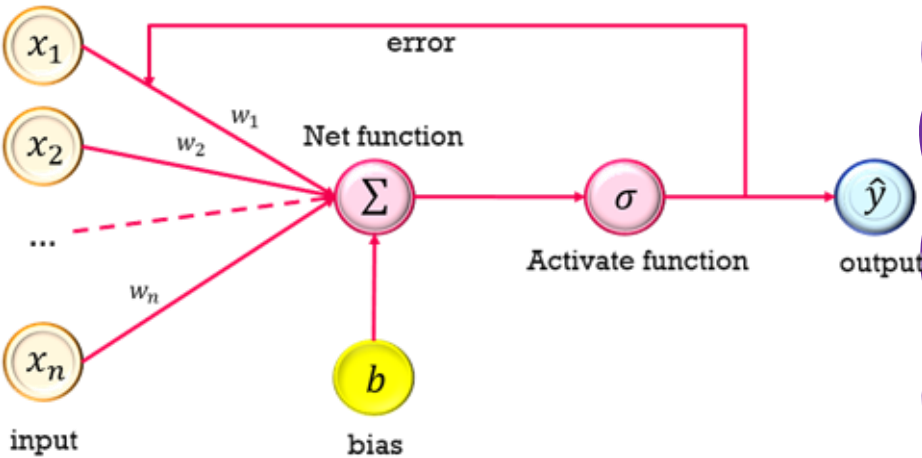
Without threshold

$$h(x) = w_1x_1 + w_2x_2 + \dots$$

Minimize the error

$$\min E(w_1, \dots, w_n) \\ \min \frac{1}{2} \sum_i (y_i - h_i)^2$$

Gradient Descent Learning



recall: $\min E(w_1, \dots, w_n) = \min \frac{1}{2} \sum_i (y_i - h_i)^2$

then,

$$\Delta w = -\alpha \nabla E[w]$$

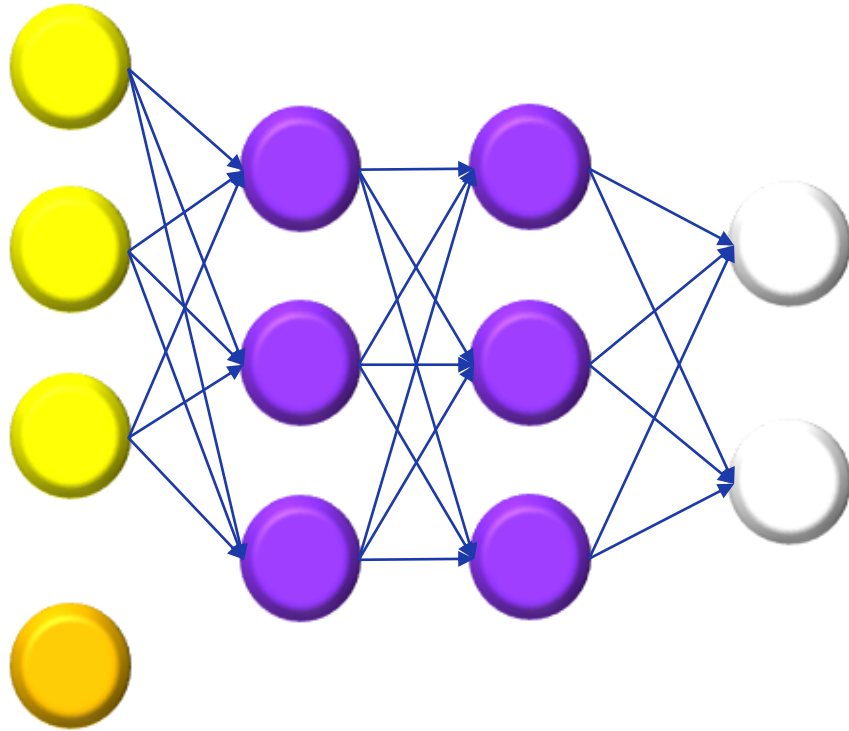
$$\Delta w = -\alpha \frac{\partial}{\partial w_i} E(w_i)$$

$$\Delta w = -\alpha \frac{\partial}{\partial w_i} \left(\frac{1}{2} \sum_i (y_i - h_i)^2 \right)$$

$$\Delta w = -\alpha \frac{\partial}{\partial w_i} \left(\frac{1}{2} \sum_i \left(y_i - \sum_j x_{ij} w_j \right)^2 \right)$$

$$\Delta w = \alpha \sum_i \left(y_i - \sum_j x_{ij} w_j \right) (x_{ij})$$

Multilayer Perceptron



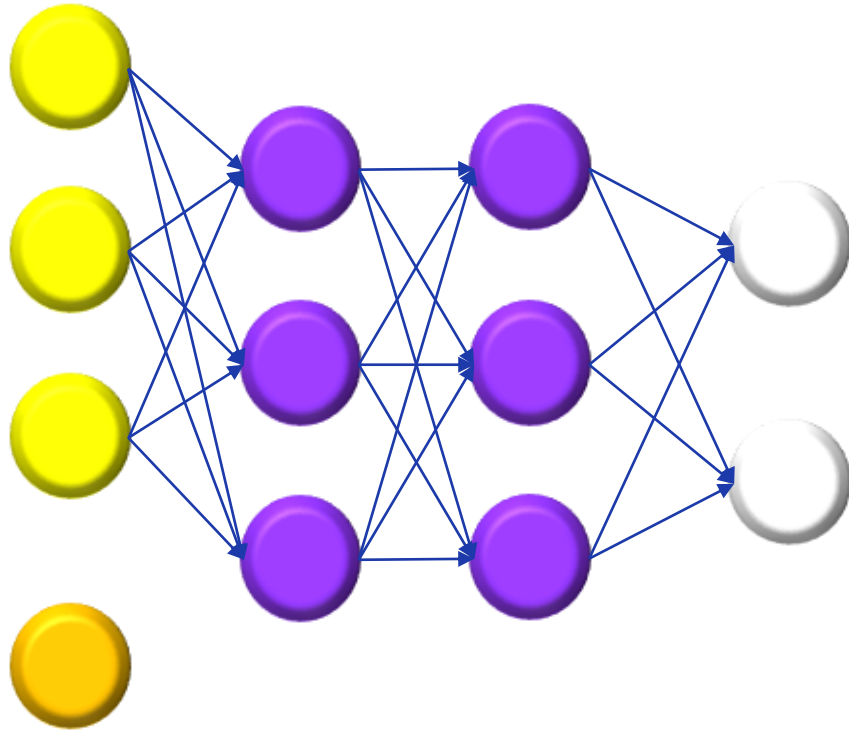
Input

hidden

output

- **Single perceptron** only capable to express **linear decision surfaces**
- **Multilayer networks** are able to express a rich variety of **nonlinear decision surfaces**

Design output layer



Input

hidden

output

Regression : identity function

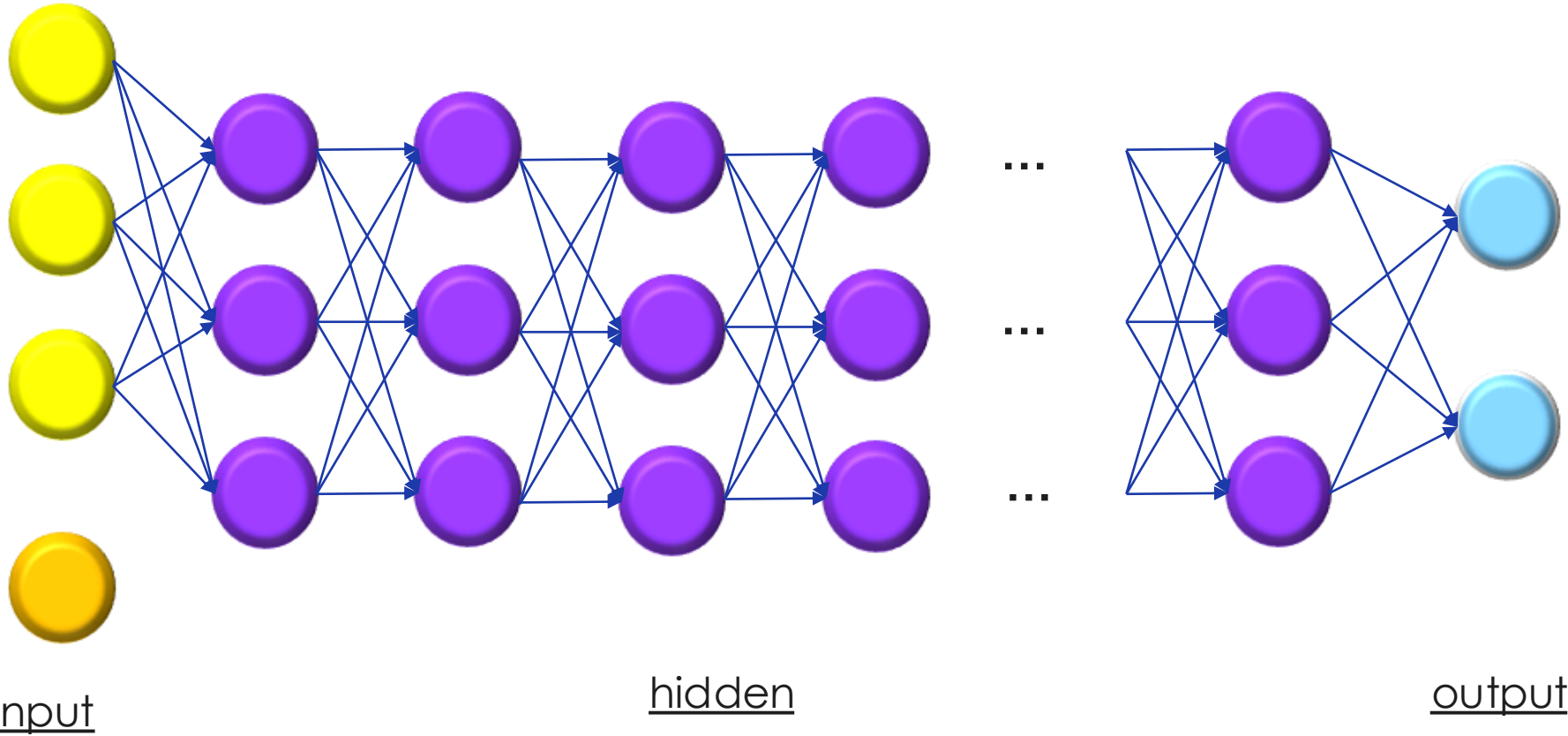
Binary classification : sigmoid function

Multiclass classification : softmax function

$$h(x, w) = \frac{e^{net_k}}{\sum_{\ell} e^{net_k}}$$

- **k binary classification** : k sigmoid function for each case

Go deeper...



Batch size:

- Number of training samples the networks evaluates per weight update step
- Cannot pass the entire dataset into the neural networks at once
- **Mini-batch** : better generalization empirically.
- **Large-batch** : more computational speed

Epoch:

- An epoch represents one iteration over the entire dataset.
- ex: 1000 dataset and a batch size 100 then an epoch has $10000/100 = 100$ iteration.

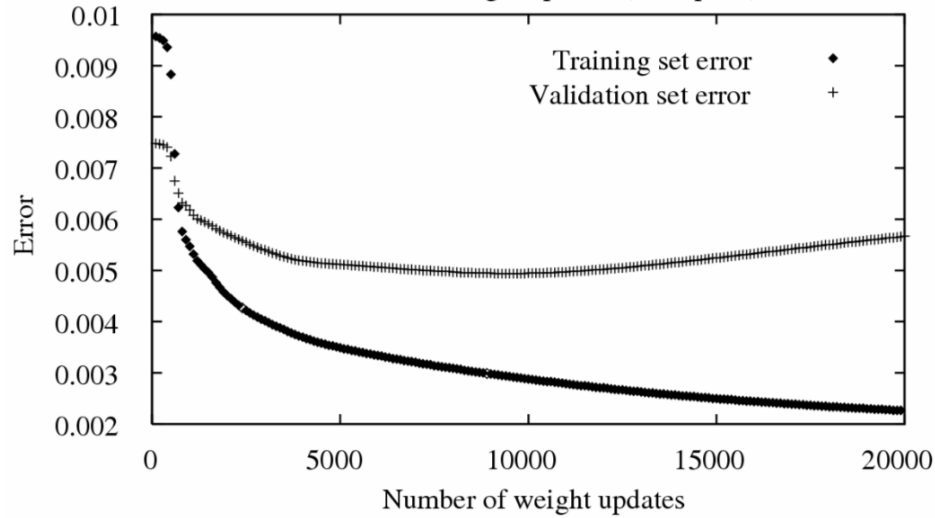
Other Learning parameters:

- **Learning rate α**
- **Momentum**
- **Etc.**

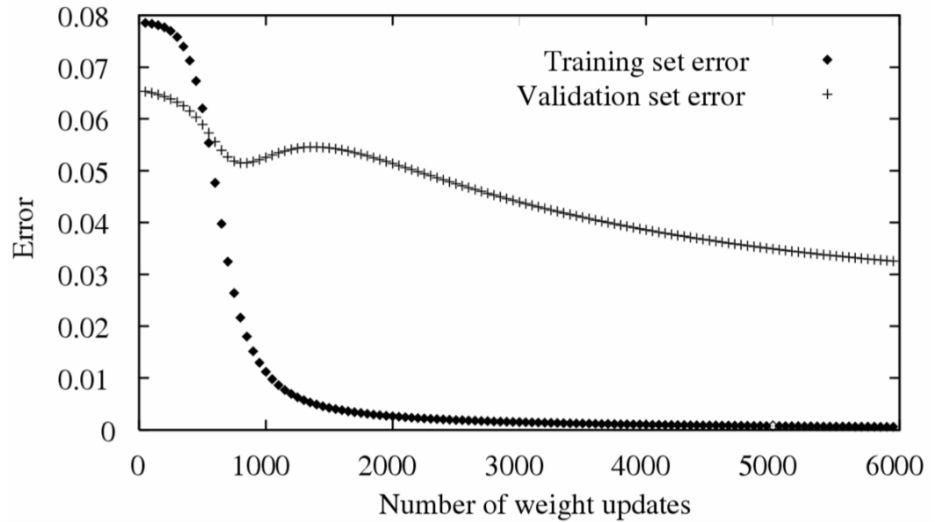
Overfitting...



Error versus weight updates (example 1)



Error versus weight updates (example 2)



Dealing with overfitting...



- **Early Stopping** : stop training when performance on the validation set decrease.

Note : validation set $\neq$ testing set

- **Regularization – Dropout** : Randomly remove some nodes in the networks

Note : usually $p \geq 0.5$ where p is probability to keep the nodes and input layers p should be much higher.

Dealing with overfitting...



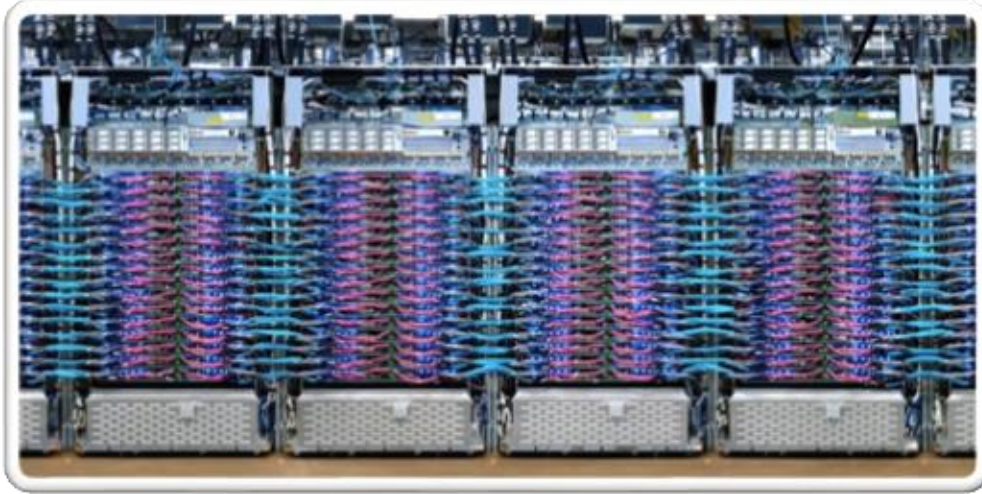
- **Regularization – weight decay** : weight penalty
 - **L1-penalty** : penalize absolute weight
 - **L2 penalty** : penalize squared weight
- **Normalization** :
 - **Networks normalization** : according to mean and standard deviation
 - **Batch normalization** : normalize hidden layer inputs to mini-batch mean and variance, reduce impact of earlier layers on later layers

Deep learning tools...



- Mark 1 Perceptron – 1960
- Torch – 2002
- CUDA – 2007
- Theano – 2008
- Caffe – 2014
- DistBelief – 2011
- TensorFlow 0.1 – 2015
- PyTorch 0.1 – 2017
- TensorFlow 0.2 -- 2019

Hardware...



- **CPU** :
Serial, general purpose, everyone has one
- **GPU** :
Parallelizable, still general purpose
- **TPU** :
Custom ASIC (Application-Specific Integrated Circuit) by Google, specialized for machine learning, low precision

Lets play with neural networks



<http://playground.tensorflow.org>

Discussion...



Q n A